



Python Matplotlib Visualisering

Statistiske plots fra CSV og live-visualisering med `plt.ion()`

Læringsmål

- ✓ Indlæse en CSV-fil med Pandas og visualisere data som linjeplot
- ✓ Lave et statistisk plot over tankniveaudata fra Emulate3D
- ✓ Bruge `plt.ion()` til live-opdaterende graf under køretid
- ✓ Beregne glidende gennemsnit med `.rolling().mean()`
- ✓ Annotere og eksportere grafer til dokumentation

Indhold

1	Introduktion til Matplotlib	2
2	Statisk plot fra CSV (Emulate3D tankdata)	3
2.1	Eksempel CSV-struktur	3
2.2	Indlæs og plot	3
2.3	Glidende gennemsnit ovenpå rå data	4
2.4	Annotering og grænselinjer	5
3	Live-plot med <code>plt.ion()</code>	6
3.1	Simpelt live-plot fra en liste	6
3.2	Live-plot med PLC-data via <code>snap7</code>	7
4	Eksport af grafer	8
5	Oversigt over pyplot-funktioner	9
6	Øvelser	9

1 | Introduktion til Matplotlib

`matplotlib.pyplot` er det centrale Python-bibliotek til visualisering. Det bruges til at oprette alt fra simple linjeplots til komplekse figursammenstillinger.

To visningsmåder

Statisk	Alle datapunkter er tilgængelige på forhånd (fra CSV-fil eller liste). Plottet bygges og vises med <code>plt.show()</code> når alt er klar.
Dynamisk	Data ankommer løbende (fra PLC, sensor eller løkke). <code>plt.ion()</code> aktiverer interaktiv tilstand — grafen opdateres i realtid uden at blokere koden.

Kode 0 — Enkelt introduktionsplot

```
1 import matplotlib.pyplot as plt
2
3 x = [0, 1, 2, 3, 4, 5]
4 y = [10, 25, 18, 30, 22, 35]
5
6 plt.plot(x, y, color="royalblue", linewidth=2, label="Måling")
7 plt.title("Simpelt linjeplot")
8 plt.xlabel("Tid (s)")
9 plt.ylabel("Værdi")
10 plt.grid(True)
11 plt.legend()
12 plt.tight_layout()
13 plt.show()
```

Funktion	Beskrivelse
<code>plt.plot(x, y)</code>	Tegn linjeplot
<code>plt.title("...")</code>	Titel på figuren
<code>plt.xlabel/ylabel</code>	Aksetekster
<code>plt.grid(True)</code>	Vis gitter
<code>plt.legend()</code>	Vis signaturforklaring
<code>plt.tight_layout()</code>	Undgå overlappende tekst
<code>plt.show()</code>	Vis figuren (blokerende)

2 | Statisk plot fra CSV (Emulate3D tankdata)

En typisk CSV-logfil fra Emulate3D indeholder tidsstempler og måleværdier. Vi indlæser den med Pandas og visualiserer den med Matplotlib.

2.1 Eksempel CSV-struktur

Eksempel: tankniveau_log.csv

Tidsstempel	Tankniveau
2024-06-01 12:00:00	45.2
2024-06-01 12:00:01	47.8
2024-06-01 12:00:02	50.1
2024-06-01 12:00:03	49.4

Filen er oprettet i Dag07 ved brug af `csv.writer` og `datetime.now()`.

2.2 Indlæs og plot

Kode 1 — Statisk plot af tankniveau fra CSV

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Indlæs CSV-filen
5 df = pd.read_csv("tankniveau_log.csv")
6
7 # Konverter tidsstempel til datetime hvis det er en streng
8 df["Tidsstempel"] = pd.to_datetime(df["Tidsstempel"])
9
10 # Plot
11 plt.figure(figsize=(10, 5))
12 plt.plot(df["Tidsstempel"], df["Tankniveau"],
13         color="royalblue", linewidth=2, label="Tankniveau")
14
15 plt.title("Tankniveau over tid (Emulate3D)")
16 plt.xlabel("Tidsstempel")
17 plt.ylabel("Tankniveau (%)")
18 plt.ylim(0, 100)
19 plt.grid(True)
20 plt.legend()
21 plt.xticks(rotation=45)
22 plt.tight_layout()
23 plt.savefig("tankniveau_statisk.png", dpi=300)
24 plt.show()
```

Tip — `pd.to_datetime()`

Pandas læser tidsstempler som tekst som standard. Kald `pd.to_datetime()` for at omveksle dem til rigtige datetime-værdier — det giver korrekt formatering på x-aksen og muliggør tidsberegninger.

2.3 Glidende gennemsnit ovenpå rå data

Kode 2 — Rå data + glidende gennemsnit

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 df = pd.read_csv("tankniveau_log.csv")
5 df["Tidsstempel"] = pd.to_datetime(df["Tidsstempel"])
6
7 # Beregn glidende gennemsnit med vindue paa 5 maelinger
8 df["Glidende"] = df["Tankniveau"].rolling(window=5,
9     → min_periods=1).mean()
10
11 plt.figure(figsize=(10, 5))
12 plt.plot(df["Tidsstempel"], df["Tankniveau"],
13     color="steelblue", alpha=0.5, linewidth=1, label="Rå data")
14 plt.plot(df["Tidsstempel"], df["Glidende"],
15     color="darkorange", linewidth=2, linestyle="--",
16     label="Glidende gns. (n=5)")
17
18 plt.title("Tankniveau med glidende gennemsnit")
19 plt.xlabel("Tidsstempel")
20 plt.ylabel("Tankniveau (%)")
21 plt.ylim(0, 100)
22 plt.grid(True)
23 plt.legend()
24 plt.xticks(rotation=45)
25 plt.tight_layout()
26 plt.savefig("tankniveau_rolling.png", dpi=300)
27 plt.show()
```

2.4 Annotering og grænselinjer

Kode 3 — Annotation og tarskellinjer

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 df = pd.read_csv("tankniveau_log.csv")
5 df["Tidsstempel"] = pd.to_datetime(df["Tidsstempel"])
6
7 fig, ax = plt.subplots(figsize=(10, 5))
8 ax.plot(df["Tidsstempel"], df["Tankniveau"],
9         color="royalblue", linewidth=2, label="Tankniveau")
10
11 # Markeer maksimum
12 idx_max = df["Tankniveau"].idxmax()
13 ax.annotate(
14     f'Max: {df["Tankniveau"][idx_max]:.1f}%',
15     xy=(df["Tidsstempel"][idx_max], df["Tankniveau"][idx_max]),
16     xytext=(10, 10), textcoords="offset points",
17     arrowprops=dict(arrowstyle="->", color="black"),
18     fontsize=9
19 )
20
21 # Grænselinjer for setpoint og alarm
22 ax.axhline(y=80, color="orange", linestyle="--", linewidth=1.5,
23            label="Setpoint (80%)")
24 ax.axhline(y=20, color="red", linestyle=":", linewidth=1.5,
25            label="Alarm (20%)")
26
27 ax.set_title("Tankniveau med setpoint og alarm")
28 ax.set_xlabel("Tidsstempel")
29 ax.set_ylabel("Tankniveau (%)")
30 ax.set_ylim(0, 100)
31 ax.grid(True)
32 ax.legend()
33 plt.xticks(rotation=45)
34 plt.tight_layout()
35 plt.savefig("tankniveau_annoteret.png", dpi=300)
36 plt.show()
```

3 | Live-plot med `plt.ion()`

`plt.ion()` (interactive on) aktiverer Matplotlibs interaktive tilstand. Det betyder at grafen opdateres løbende mens koden kører — perfekt til realtidvisning af PLC-data eller simulerede forløb.

Hvordan virker `plt.ion()`?

<code>plt.ion()</code>	Aktiverer interaktiv tilstand (kældes een gang ved start).
<code>plt.pause(n)</code>	Opdaterer grafen og venter <i>n</i> sekunder — erstatter <code>time.sleep()</code> i løkken.
<code>ax.cla()</code>	Rydder akserne før næste tegning (undgår ophobning).
<code>plt.ioff()</code>	Deaktiverer interaktiv tilstand når løkken stopper.
<code>plt.show()</code>	Holder vinduet åbent efter løkken.

3.1 Simpelt live-plot fra en liste

Kode 4 — Live-plot med simulerede værdier

```
1 import matplotlib.pyplot as plt
2 import random
3 import time
4
5 # Aktiver interaktiv tilstand
6 plt.ion()
7 fig, ax = plt.subplots(figsize=(9, 4))
8
9 x_data = []
10 y_data = []
11
12 for i in range(50):
13     x_data.append(i)
14     y_data.append(random.uniform(40, 90)) # Simuleret tankniveau
15
16     ax.cla() # Ryd akserne
17     ax.plot(x_data, y_data, color="royalblue", linewidth=2)
18     ax.set_title("Live tankniveau (simuleret)")
19     ax.set_xlabel("Måling nr.")
20     ax.set_ylabel("Tankniveau (%)")
21     ax.set_ylim(0, 100)
22     ax.grid(True)
23
24     plt.pause(0.1) # Opdater og vent 0.1 sek.
25
26 plt.ioff()
27 plt.show()
```

3.2 Live-plot med PLC-data via snap7

Kode 5 — Live-plot fra PLC (snap7 + plt.ion())

```

1 import matplotlib.pyplot as plt
2 import snap7
3 from snap7.util import get_real
4 from datetime import datetime
5
6 # ===== PLC opsætning =====
7 PLC_IP      = "192.168.0.1"
8 RACK        = 0
9 SLOT        = 1
10 DB_NUMBER   = 1
11 TANK_OFFSET = 0
12 TANK_SIZE   = 4
13
14 # ===== PLC forbindelse =====
15 client = snap7.client.Client()
16 client.connect(PLC_IP, RACK, SLOT)
17
18 # ===== Live-plot opsætning =====
19 plt.ion()
20 fig, ax = plt.subplots(figsize=(10, 5))
21
22 timestamps = []
23 niveauer   = []
24
25 try:
26     while True:
27         # ===== Læs input =====
28         raw = client.db_read(DB_NUMBER, TANK_OFFSET, TANK_SIZE)
29
30         # ===== Logik =====
31         niveau = round(get_real(raw, 0), 2)
32         ts     = datetime.now().strftime("%H:%M:%S")
33         timestamps.append(ts)
34         niveauer.append(niveau)
35
36         # Behold kun de seneste 60 maelinger
37         if len(timestamps) > 60:
38             timestamps.pop(0)
39             niveauer.pop(0)
40
41         # ===== Opdater graf =====
42         ax.cla()
43         ax.plot(timestamps, niveauer, color="royalblue", linewidth=2)
44         ax.axhline(y=80, color="orange", linestyle="--",
45                    label="Setpoint")
46         ax.axhline(y=20, color="red", linestyle=":", label="Alarm")
47         ax.set_title("Live tankniveau fra PLC")
48         ax.set_xlabel("Tid")
49         ax.set_ylabel("Tankniveau (%)")
50         ax.set_ylim(0, 100)
51         ax.legend()
52         ax.grid(True)
53         plt.xticks(rotation=45)
54         plt.tight_layout()

```


⚠ OBS — Vinduesstyring

På nogle systemer kræver `plt.ion()` et interaktivt backend som `TkAgg` eller `Qt5Agg`. Hvis vinduet ikke opdateres, afprøv:

```
import matplotlib.pyplot as plt
plt.style.use("TkAgg") # foer import af pyplot
```

4 | Eksport af grafer

Grafer kan gemmes som billedfil med `plt.savefig()`. Kald det **før** `plt.show()` — ellers gemmes en tom figur.

📄 Kode 6 — Gem graf i PNG og PDF

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3 from datetime import datetime
4
5 df = pd.read_csv("tankniveau_log.csv")
6 df["Tidsstempel"] = pd.to_datetime(df["Tidsstempel"])
7
8 plt.figure(figsize=(10, 5))
9 plt.plot(df["Tidsstempel"], df["Tankniveau"],
10         color="royalblue", linewidth=2)
11 plt.title("Tankniveau --- eksporteret graf")
12 plt.xlabel("Tidsstempel")
13 plt.ylabel("Tankniveau (%)")
14 plt.grid(True)
15 plt.tight_layout()
16
17 # Gem med tidsstempel i filnavnet
18 ts = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
19 plt.savefig(f"{ts}_tankniveau.png", dpi=300) # Høj oplosning
20 plt.savefig(f"{ts}_tankniveau.pdf")         # Vektorgrafik
21 plt.show()
```

Format	Fordel	Typisk brug
.png	Simpelt billedformat, bred støtte	Rapporter, slides
.pdf	Vektorgrafik, skalerer uden tab	Dokumentation, print
.svg	Vektorgrafik til web	HTML-sider
.jpg	Lille filstørrelse	Preview, deling

5 | Oversigt over pyplot-funktioner

Funktion	Beskrivelse
<code>plt.ion()</code>	Aktiver interaktiv (live) tilstand
<code>plt.ioff()</code>	Deaktiver interaktiv tilstand
<code>plt.pause(n)</code>	Opdater graf + vent n sekunder
<code>ax.cla()</code>	Ryd aksens indhold
<code>plt.plot(x, y)</code>	Linjeplot
<code>plt.axhline(y=n)</code>	Vandret linje ved værdi n
<code>plt.axvline(x=n)</code>	Lodret linje ved værdi n
<code>plt.annotate(...)</code>	Tekst og pil ved et datapunkt
<code>plt.savefig(f, dpi=300)</code>	Gem figur som billedfil
<code>pd.to_datetime()</code>	Konverter streng til datetime
<code>.rolling(window=n).mean()</code>	Glidende gennemsnit

6 | Øvelser

☰ Øvelsesopgaver

- Statisk plot fra CSV**
Indlæs din CSV-logfil fra Dag07 (tankniveau med tidsstempler) og lav et statisk linjeplot med titler, aksetekster og gitter.
- Glidende gennemsnit**
Tilføj et glidende gennemsnit (`window=5`) til plottet fra øvelse 1. Vis både rå data og glidende gennemsnit i samme graf.
- Annotation og grænselinjer**
Marker maksimum-værdien med en pil og tilføj vandrette linjer for setpoint (80%) og alarm (20%) til plottet.
- Live-plot med simulerede data**
Brug `plt.ion()` og en for-løkke med `random.uniform()` til at lave et live-opdaterende plot.
- Live-plot fra PLC**
Udvid Kode 5 til også at vise det glidende gennemsnit i live-grafen og gem det endelige plot som PNG når løkken stopper.
- Eksport**
Gem alle dine grafer som både `.png` (300 dpi) og `.pdf` med et tidsstempel i filnavnet.